Two Pointer Technique Study Guide

What is Two Pointer Technique?

The Two Pointer Technique is an algorithmic approach that uses two pointers (indices) to traverse through data structures, typically arrays or strings. These pointers can move in the same direction or opposite directions depending on the problem requirements.

Types of Two Pointer Patterns

1. Opposite Direction Pointers

Two pointers start from opposite ends and move towards each other.

```java
public class OppositeDirection {
    public boolean isPalindrome(String s) {
        int left = 0;
        int right = s.length() - 1;

        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}
```

2. Same Direction Pointers (Fast and Slow)

Two pointers start from the same position but move at different speeds.

```java
public class SameDirection {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;

        int slow = 0;
        for (int fast = 1; fast < nums.length; fast++) {
            if (nums[fast] != nums[slow]) {
                slow++;
                nums[slow] = nums[fast];
            }
        }
        return slow + 1;
    }
}
```

3. Sliding Window Pointers

Two pointers maintain a window that can expand or contract based on conditions.

```java
public class SlidingWindow {
    public int maxSubarraySum(int[] nums, int k) {
        int left = 0, right = 0;
        int currentSum = 0;
        int maxSum = Integer.MIN_VALUE;

        while (right < nums.length) {
            currentSum += nums[right];

            if (right - left + 1 == k) {
                maxSum = Math.max(maxSum, currentSum);
                currentSum -= nums[left];
                left++;
            }
            right++;
        }
        return maxSum;
    }
}
```

Common Two Pointer Problems and Solutions

Problem 1: Two Sum in Sorted Array

```java
public class TwoSum {
    public int[] twoSum(int[] numbers, int target) {
        int left = 0;
        int right = numbers.length - 1;

        while (left < right) {
            int sum = numbers[left] + numbers[right];
            if (sum == target) {
                return new int[]{left + 1, right + 1}; // 1-indexed
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }
        return new int[]{-1, -1}; // No solution found
    }
}
```

Problem 2: Container With Most Water

```java
public class ContainerWithMostWater {
    public int maxArea(int[] height) {
        int left = 0;
        int right = height.length - 1;
        int maxWater = 0;

        while (left < right) {
            int width = right - left;
            int currentWater = Math.min(height[left], height[right]) * width;
            maxWater = Math.max(maxWater, currentWater);

            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return maxWater;
    }
}
```

Problem 3: Remove Element

```java
public class RemoveElement {
    public int removeElement(int[] nums, int val) {
        int slow = 0;

        for (int fast = 0; fast < nums.length; fast++) {
            if (nums[fast] != val) {
                nums[slow] = nums[fast];
                slow++;
            }
        }
        return slow;
    }
}
```

Problem 4: Valid Palindrome

```java
public class ValidPalindrome {
    public boolean isPalindrome(String s) {
        int left = 0;
        int right = s.length() - 1;

        while (left < right) {
            while (left < right && !Character.isLetterOrDigit(s.charAt(left))) {
                left++;
            }
            while (left < right && !Character.isLetterOrDigit(s.charAt(right))) {
                right--;
            }

            if (Character.toLowerCase(s.charAt(left)) !=
                Character.toLowerCase(s.charAt(right))) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}
```

Problem 5: Three Sum

```java
import java.util.*;

public class ThreeSum {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue; // Skip duplicates

            int left = i + 1;
            int right = nums.length - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));

                    while (left < right && nums[left] == nums[left + 1]) left++;
                    while (left < right && nums[right] == nums[right - 1]) right--;

                    left++;
                    right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}
```

Time and Space Complexity

Time Complexity

- Most two-pointer problems: **O(n)** where n is the size of the array
- Problems requiring sorting first: **O(n log n)** due to sorting

Space Complexity

- Generally **O(1)** extra space (excluding input and output)
- In-place modifications without additional data structures

When to Use Two Pointer Technique

1. **Sorted Arrays**: When dealing with sorted arrays and need to find pairs or triplets
2. **Palindrome Problems**: Checking if strings or arrays are palindromes
3. **Array Modification**: Removing elements or duplicates in-place
4. **Sliding Window**: Finding subarrays with specific properties
5. **Linked Lists**: Detecting cycles or finding middle elements

Key Tips for Implementation

1. **Initialize Pointers Correctly**: Choose starting positions based on problem requirements
2. **Define Movement Logic**: Decide when to move which pointer
3. **Handle Edge Cases**: Empty arrays, single elements, all duplicates
4. **Boundary Conditions**: Ensure pointers don't go out of bounds
5. **Optimization**: Skip unnecessary iterations when possible

---

Quiz Questions

Question 1: What is the time complexity of the two-pointer approach for finding a pair with a given sum in a sorted array? a) $O(n^2)$ b) O(n log n) c) O(n) d) O(1)

Answer: c) O(n)

Question 2: In the two-pointer technique for palindrome checking, when do we move the pointers? a) Only move left pointer b) Only move right pointer c) Move both pointers towards each other d) Move pointers in the same direction

Answer: c) Move both pointers towards each other

Question 3: Which of the following problems is NOT typically solved using two pointers? a) Two Sum in sorted array b) Binary Search c) Remove duplicates from sorted array d) Valid Palindrome

Answer: b) Binary Search

Question 4: In the sliding window variant of two pointers, what happens when the window condition is violated? a) Both pointers move forward b) Only right pointer moves forward c) Left pointer moves

forward to shrink window d) Both pointers reset to start

Answer: c) Left pointer moves forward to shrink window

Question 5: What is the space complexity of most two-pointer algorithms? a) O(n) b) O(log n) c) $O(n^2)$ d) O(1)

Answer: d) O(1)

Practice Coding Questions

Question 1: Given a sorted array of integers and a target sum, write a Java method to find all unique pairs of numbers that add up to the target sum. Return the pairs as a list of integer arrays.

Question 2: Implement a method to reverse only the vowels in a given string using the two-pointer technique. For example, "hello" should become "holle".

Question 3: Given an array of integers, write a method to move all zeros to the end while maintaining the relative order of non-zero elements. Use the two-pointer approach and modify the array in-place.

Question 4: Write a Java method to find the longest substring without repeating characters using the sliding window (two-pointer) technique.

Question 5: Given a sorted array that may contain duplicates, implement a method to remove all duplicates such that each element appears at most twice. Return the new length of the array.